

האוניברסיטה העברית בי"ס להנדסה ולמדעי המחשב

בחינה בקורס מבוא למדעי המחשב (67101)

תאריך: 21.1.2003
משך הבחינה: שעתיים וחצי

מועד א' - סמסטר א' - תשס"ג - 2002-2003
המורה: פרופ ג'ף רוזנשיין

הנחיות:

- יש לענות על 3 מתוך 4 השאלות (משקל כל השאלות זהה).
- שפת התכנות בה עליכם להשתמש היא **Java**.
- הקפידו הקפדה יתרה על הסדר ועל כתב יד קריא. בחינה בלתי קריאה עלולה שלא לזכות את כותבה במלוא הנקודות!
- שימו לב:
 - יש להתחיל את התשובה לכל שאלה בעמוד חדש, ולציין בראש העמוד בצורה ברורה את מספר השאלה.
 - יש למחוק טיוטות באופן ברור.
 - יש להשאיר שוליים.
 - אסור להשתמש בעט אדום.
- הקפידו על סגנון תכנותי נאות: כתבו באופן מדולרי, אל תכפילו קוד וכו'.
- תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית).
- כפתרון מיטבי לשאלה יחשב פתרון פשוט ויעיל ככל האפשר.
- אין להשתמש במבני נתונים מלבד אלה שהוגדרו בכל שאלה.

שאלה 1

הגדרת בעיית הבחירה (selection problem):

יהי `set` מערך שמוגדר באופן הבא:
`int[] set = new int[N];`
יהי `k` מספר כך ש:
`0 ≤ k < N`
בכדי לפתור את בעיית הבחירה עלינו למצוא איבר `e` כך שאם היינו ממיינים את `set` אזי
היה מתקיים:
`set[k] == e`
בכדי לפתור את בעיית הבחירה בצורה היעילה ביותר נהוג להשתמש בפונקציה:
`static int partition(int set[], int lo, int hi);`
כפי שהוגדרה בכיתה כאשר נלמד האלגוריתם `quicksort`.

א. הסבר בקצרה מה מטרת הפונקציה `partition` ומה היא מחזירה (אל תסביר כיצד האלגוריתם פועל). תן דוגמא בה `lo ≠ 0` וכן `hi ≠ (set.length - 1)` שתשכנע כי אתה מבין מה מחזירה הפונקציה. (4 נק')

ב. כתוב פונקציה סטטית בשם `selection` (השייכת ל- `class` ללא `data-members`) שמקבלת את `set` ואת `k` ופותרת את בעיית הבחירה (החזרת `e`) באמצעות `partition`. (21 נק'). הנחיות:

- מותר להניח כי: `0 ≤ k < set.length`
- יש לכתוב פונקציה יעילה ככל שניתן (שימו לב שאין צורך למיין את המערך ופתרון שייעשה זאת לא יקבל נקודות כלל).
- ניתן להשתמש בפונק' עזר.

ג. מהו סדר גודל זמן הריצה של האלגוריתם שכתבת בסעיף ב' במקרה הגרוע ביותר? הסבר. (3 נק').

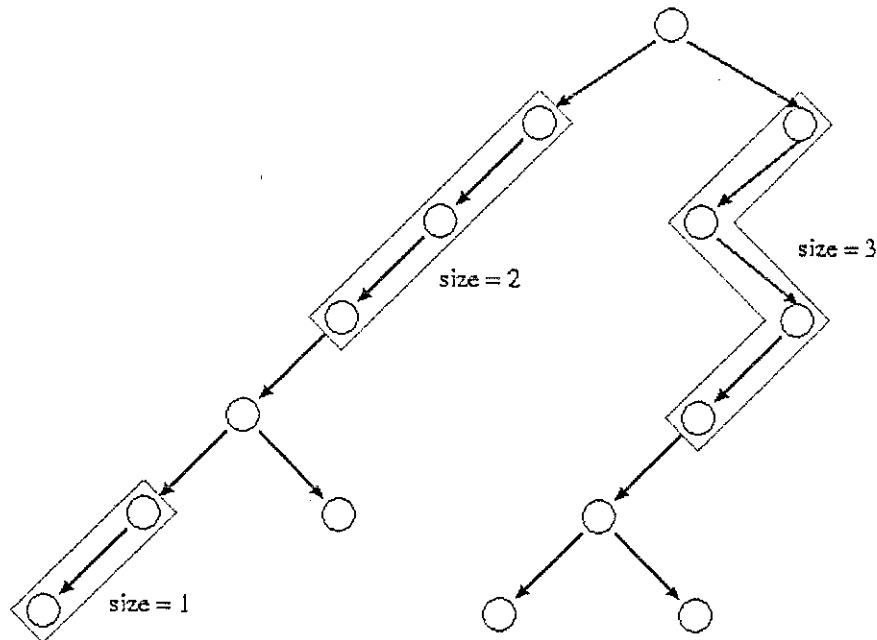
ד. מהו סדר גודל זמן הריצה של האלגוריתם שכתבת בסעיף ב' במקרה הטוב ביותר? הסבר. (5 נק').

שאלה 2

בתכנית מוגדר עץ חיפוש בינארי באמצעות המבנה:

```
class Node {
    public int data;
    public Node left;
    public Node right;
}
```

נגדיר "מקל" בעץ כמסלול אשר הקדקודים המרכיבים אותו הנם לכל היותר בעלי בן אחד. נגדיר "אורך של מקל" כמספר הצלעות המרכיבות את המקל. נגדיר שאורך המקל המקסימלי בעץ ריק הוא: 1-
לדוגמא, העץ הבא מכיל שלושה מקלות לא טריוויאליים (מקל טריוויאלי הוא מקל באורך 0) אשר לא ניתן להגדילם בעלי אורכים 1, 2 ו-3:



שימו לב שבכל עץ שאיננו ריק יש לכל הפחות מקל טריויאלי אחד (עלה).

א. כתוב פונקציה סטטית בשם stick (השייכת ל-class ללא data-members) המקבלת שורש של עץ (Node) כנ"ל, ומחזירה את אורך המקל המקסימלי בעץ. הפונקציה תחזיר 3 בעבור העץ הנ"ל. (30 נק'). הנחיות:

- הקפד על יעילות הפונקציה שכתבת.
- אסור להשתמש במבני נתונים נוספים מלבד העץ הנתון.
- אסור להוסיף פונק' ל- Node אך ניתן להשתמש בפונקציות עזר.

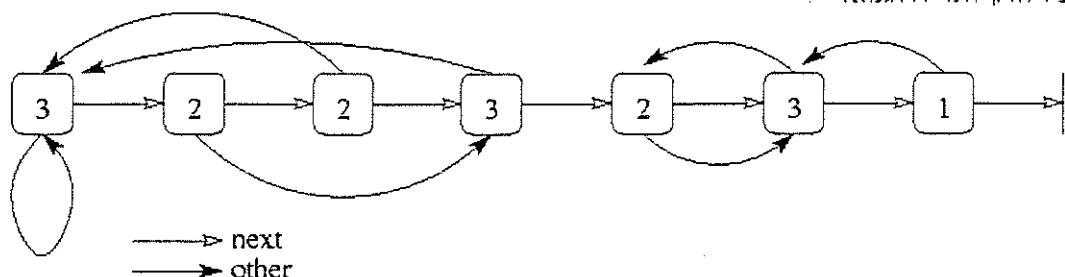
ב. מה סדר גודל זמנו הריצה של הפונקציה שכתבת במקרה הגרוע? הסבר. (3 נ').

שאלה 3

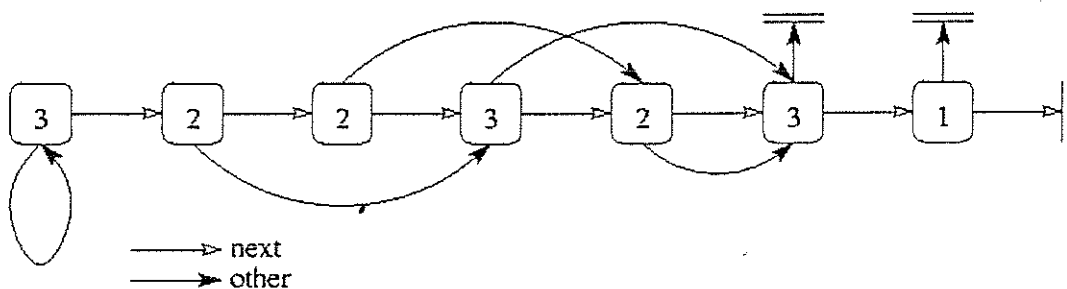
בתכנית מוגדרת רשימה משורשרת באמצעות המבנה:

```
class Node {
    public int data;
    public Node next, other;
}
```

כך שאם נתקדם מאיבר לאיבר באמצעות המצביע next אזי נסרוק את הרשימה באופן סדרתי (כמימים ימימה). המצביע other מצביע לתא כלשהו ברשימה (שאיננו null) ללא כל חוקיות. לדוגמא:



א. כתוב פונקציה סטטית בשם forward (השייכת ל- class ללא data-members) אשר מקבלת מצביע (Node) לראשה של רשימה משורשרת כנ"ל, ומשנה אותה כך שכל תא שבו other הצביע "אחורה", כעת יצביע "קדימה":
 i. לתא הבא עם ערך data זהה (אם קיים), או
 ii. ל- null (אם לא קיים).
 לדוגמא, הרשימה הנ"ל תעודכן באופן הבא:



(28 נק'). הנחיות:

- הקפד על יעילות הפונקציה שכתבת.
- אסור להשתמש במבני נתונים נוספים מלבד הרשימה הנתונה.
- אסור להוסיף פונק' ל- Node אך ניתן ואף רצוי להשתמש בפונקציות עזר.

ב. מה סדר גודל זמן הריצה של הפונקציה שכתבת במקרה הגרוע? הסבר. (5 נ').

שאלה 4

פולינדרום נומרי הינו מספר שלם בעל התכונה הבאה: אין זה משנה אם קוראים אותו משמאל לימין או מימין לשמאל שכן מתקבלת אותה תוצאה. לדוגמא, כל המספרים הבאים הינם פולינדרומים נומריים: 121, 1221, 1234321, 33333, 7, 77

זרע של פולינדרום נומרי הינו מספר s_0 שניתן להגיע ממנו לפולינדרום נומרי באמצעות התהליך הבא:

- יהי t_i המספר שהתקבל מהיפוך הספרות של s_i (בתחילת התהליך $i=0$).
- אם $s_i = t_i$ (כלומר s_i הוא פולינדרום נומרי) אזי קובעים ש- s_0 מהווה זרע של פולינדרום ומפסיקים את התהליך.
- אחרת, קובעים: $s_{i+1} = s_i + t_i$ וממשיכים בתהליך עבור s_{i+1} .

לדוגמא: $s_0 = 165$ מהווה זרע-של-פולינדרום-נומרי כיוון ש s_3 מהווה פולינדרום-נומרי:

$$\begin{array}{ll} s_0 = 165 & (t_0 = 561) \\ s_1 = s_0 + t_0 = 165 + 561 = 726 & (t_1 = 627) \\ s_2 = s_1 + t_1 = 726 + 627 = 1353 & (t_2 = 3531) \\ s_3 = s_2 + t_2 = 1353 + 3531 = 4884 & (t_3 = 4884 = s_3) \end{array}$$

מסתבר שיש הרבה זרעים כנ"ל, שכן פרט ל- 5996 יוצאי דופן, כל המספרים האי שליליים הקטנים מ- 100,000 מהווים זרעים של פולינדרומים נומריים.

N-זרע של פולינדרום נומרי הינו זרע של פולינדרום נומרי שבעבורו כל s_i בתהליך הנ"ל מקיים: $s_i \leq N$ (לכל i).

ע"י שימוש בעקרונות ה- top down design, כתוב מחלקה ב- Java אשר הרצת ה- main שלה תגרום להדפסת כל המספרים בין 10 ל- 100,000 שאִינֵם N-זרע של פולינדרום נומרי (נקבע כי בתוכניתכם, $N = 180,000,000$). בעבור כל מספר s_0 שאינו N-זרע עליכם להדפיס את השורה:

s_0 may not be a good seed: it was checked until s_k

כאשר s_k הינו המספר שגרם לסיום התהליך ($s_k > N$). (33 נק').

הנחיות:

- אין צורך לבצע אופטימיזציה שתמנע מ"בדיקות מיותרות". למשל, אחרי שהוכחנו (בדוגמא למעלה) ש- 165 הוא N-זרע, אם היינו רוצים (ואנחנו לא רוצים), היינו יכולים לדלג לאחר מכן על בדיקת המספרים: 561, 627, 726, 1353, 3531 ו- 4884 (שכן למעשה הוכחנו כבר שגם מספרים אלה הינם N-זרעים).
 - אין לאכסן את המספרים במערכים או במבני נתונים אחרים (ואין גם צורך).
 - שימו לב שאם s_0 הוא פולינדרום-נומרי הרי שהוא גם זרע-של-פולינדרום-נומרי.
 - לידע כללי בלבד: ה- int הגדול ביותר ב- Java הוא:
- `java.lang.Integer.MAX_VALUE = 2,147,483,647`
- אין צורך לטפל ב- integer-overflow.

בהצלחה